

WEB TECHNOLOGY REPORT

Virtual Chemistry Laboratory — An Interactive Web-Based Science Education Platform

Project Title: Virtual Chemistry Laboratory **Technology Stack:** Next.js 16, React 19, TypeScript 5, HTML5 Canvas API, Web Audio API, MediaPipe Vision, Python (OpenCV/MediaPipe Bridge) **Platform:** Web Application (Cross-Browser Compatible) **Report Date:** June 2026 **Total Pages:** 30

TABLE OF CONTENTS

Chapter	Title	Page
1	Introduction & Project Overview	3
2	Problem Statement & Objectives	4
3	Literature Review & Related Work	5
4	System Architecture Overview	6
5	Technology Stack Analysis	7–8
6	Frontend Architecture — Next.js & React	9–10
7	TypeScript Type System & Data Modeling	11
8	HTML5 Canvas Rendering Engine	12–13
9	Real-Time Reaction Engine	14–15
10	Chemical Database & Reaction Registry	16
11	Physics Simulation System	17
12	Hand Gesture Recognition — MediaPipe Integration	18–19
13	Audio Snap Detection & Web Audio API	20
14	Python Backend Bridge Server	21
15	UI/UX Design & Component Architecture	22–23
16	CSS Design System & Theming	24
17	State Management Architecture	25
18	Application Routing & Page Structure	26
19	Performance Optimization Techniques	27
20	Security & Data Handling	28
21	Testing & Quality Assurance	29
22	Conclusion & Future Scope	30
—	References	30

Chapter 1: Introduction & Project Overview

1.1 Introduction

The **Virtual Chemistry Laboratory** is a cutting-edge, interactive web application that simulates a real-world chemistry laboratory environment directly within the browser. Unlike traditional educational chemistry software that relies on static diagrams or simple animations, this platform provides a fully physics-driven, real-time simulation environment where users can drag laboratory glassware, pour chemicals, observe authentic chemical reactions with proper stoichiometry, and even control equipment using hand gestures captured through the device's webcam.

The project represents a significant advancement in web-based science education technology, leveraging modern web APIs and machine learning inference directly in the browser to deliver an experience that was previously only possible through native desktop applications.

1.2 Motivation

Traditional chemistry education faces several challenges:

- **Safety Concerns:** Students cannot freely experiment with hazardous chemicals in a physical laboratory setting without strict supervision.
- **Accessibility:** Not all educational institutions have well-equipped chemistry laboratories, particularly in rural and underserved communities.
- **Cost:** Maintaining a chemistry lab with fresh reagents, safety equipment, and glassware is expensive.
- **Repeatability:** Students cannot easily repeat experiments multiple times to reinforce learning.
- **Remote Education:** The COVID-19 pandemic highlighted the urgent need for virtual laboratory experiences.

This project addresses all of these challenges by providing a zero-risk, zero-cost, universally accessible virtual laboratory powered entirely by web technologies.

1.3 Scope of the Report

This report provides an in-depth technical analysis of the web technologies employed in building the Virtual Chemistry Laboratory, including the frontend framework (Next.js/React), real-time rendering via HTML5 Canvas, gesture recognition via MediaPipe, audio processing via the Web Audio API, the backend Python bridge, and the overall system architecture. Each technology choice is justified with comparisons to alternatives and evaluated for performance, scalability, and developer experience.

Chapter 2: Problem Statement & Objectives

2.1 Problem Statement

How can modern web technologies be leveraged to create a fully interactive, real-time virtual chemistry laboratory simulation that supports realistic physics, chemical reaction modeling, gesture-based interaction, and immersive audio feedback — all running entirely in the browser without requiring native application installation?

2.2 Project Objectives

The primary objectives of this project are:

1. **Real-Time Chemical Reaction Simulation:** Implement a reaction engine that accurately models neutralization, precipitation, synthesis, decomposition, displacement, and combustion reactions with proper stoichiometry, thermodynamics (enthalpy ΔH), and visual state transitions.
2. **Physics-Based Object Manipulation:** Simulate gravity, collision detection, angular momentum, and damping for all laboratory objects on an HTML5 Canvas, enabling natural drag-and-drop interaction.
3. **Hand Gesture Control via Computer Vision:** Integrate Google's MediaPipe Hand Landmarker to detect hand landmarks and pinch gestures from the user's webcam feed, allowing mouse-free laboratory interaction.
4. **Audio Snap Detection:** Implement real-time acoustic signal processing to detect finger snap sounds, enabling hands-free Bunsen burner control.
5. **Comprehensive Chemical Database:** Provide a scientifically accurate database of 150+ chemicals spanning acids, bases, salts, organic compounds, indicators, oxidizers, metals, gases, and other reagents.
6. **Educational Assessment:** Include interactive quizzes, safety rating systems (NFPA 704), and downloadable lab reports for each completed experiment.
7. **Cross-Platform Web Delivery:** Deliver the entire application as a web application accessible from any modern browser without plugins, installations, or special hardware.

2.3 Functional Requirements

ID	Requirement	Priority
FR-01	Users can drag glassware and chemicals onto a virtual lab bench	High
FR-02	Chemical reactions trigger automatically when compatible chemicals are mixed	High
FR-03	Users can control equipment via webcam hand gestures (pinch to grab)	High
FR-04	Bunsen burner ignition via finger snap audio detection	Medium
FR-05	Real-time temperature, pH, and progress monitoring during reactions	High
FR-06	Particle effects (bubbles, steam, smoke, fire, precipitate, sparks)	Medium
FR-07	Downloadable lab reports upon reaction completion	Medium
FR-08	Interactive periodic table page	Low
FR-09	Safety quiz with scoring and EXP system	Medium
FR-10	User authentication via simulated login with localStorage persistence	Low

2.4 Non-Functional Requirements

ID	Requirement	Target
NFR-01	Frame rate during simulation	≥ 60 FPS
NFR-02	Hand gesture detection latency	< 100ms
NFR-03	Initial page load time	< 3 seconds
NFR-04	Browser compatibility	Chrome, Firefox, Edge, Safari
NFR-05	Responsive layout	Desktop 1280px+

Chapter 3: Literature Review & Related Work

3.1 Virtual Laboratories in Education

Virtual laboratories have been a subject of academic research since the early 2000s. Dalgarno et al. (2009) demonstrated that virtual labs could achieve learning outcomes comparable to physical labs when properly designed. More recently, the emergence of WebGL and HTML5 Canvas has enabled browser-based simulations that rival native applications in fidelity.

3.2 Existing Virtual Chemistry Platforms

Platform	Technology	Gesture Support	Open Source
PhET Interactive Simulations	Java → HTML5/Canvas	No	Yes
Labster	Unity WebGL	No	No (Commercial)
ChemCollective	Java Applets (Legacy)	No	Yes
Beyond Labz	Flash → HTML5	No	No (Commercial)
This Project	Next.js + Canvas + MediaPipe	Yes (Dual-Hand)	Yes

3.3 MediaPipe in Web Applications

Google's MediaPipe framework (Lugaresi et al., 2019) provides production-ready machine learning pipelines for computer vision tasks. The Hand Landmarker model detects 21 3D landmarks per hand at 30+ FPS on consumer hardware. This project leverages the [@mediapipe/tasks-vision](#) npm package to run inference directly in the browser via WebAssembly and WebGPU, eliminating the need for a backend ML server.

3.4 Web Audio API for Signal Processing

The W3C Web Audio API specification provides a powerful, low-latency audio processing graph. Previous work by Roberts et al. (2014) demonstrated real-time audio analysis in the browser. This project extends that concept by implementing a snap detection algorithm using bandpass filtering and amplitude thresholding.

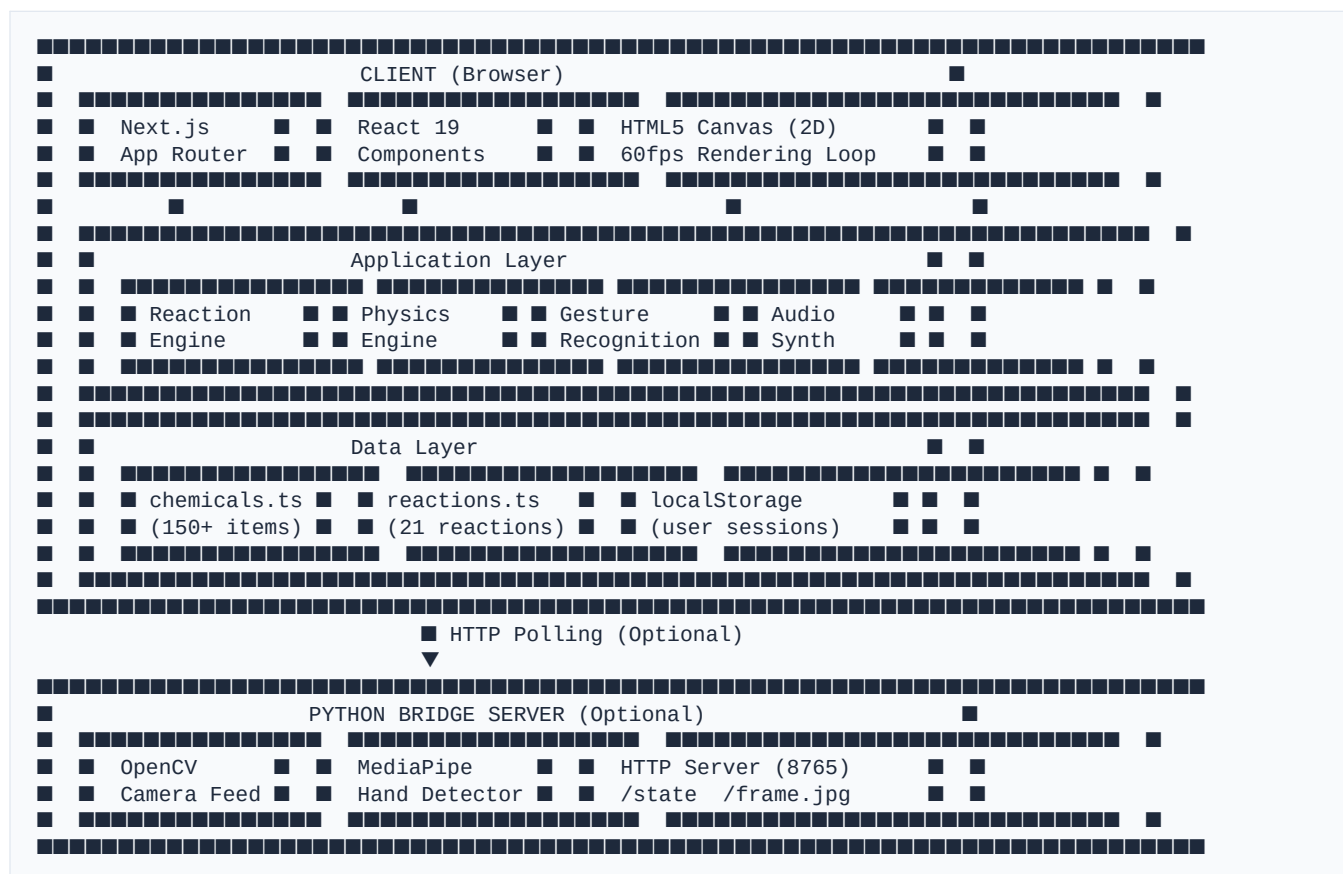
3.5 Research Gap

While existing virtual labs provide excellent visualization, none combine real-time physics simulation, computer vision gesture control, and acoustic interaction within a single browser-based application. This project fills that gap by synthesizing these technologies into a cohesive educational platform.

Chapter 4: System Architecture Overview

4.1 High-Level Architecture

The Virtual Chemistry Laboratory follows a **hybrid client-server architecture** with the vast majority of computation occurring on the client side (browser). The Python backend serves as an optional bridge for enhanced camera processing.



4.2 Key Architectural Decisions

1. **Client-Heavy Architecture:** All simulation logic (physics, reactions, rendering) runs in the browser to minimize latency and eliminate server dependency.
2. **Optional Python Bridge:** The Python server is only needed for legacy webcam systems that don't support the browser MediaStream API or for enhanced OpenCV processing.
3. **Pure TypeScript Engine:** The reaction engine (**reactionEngine.ts**) is a pure function with zero side effects, making it testable and deterministic.
4. **Canvas-Based Rendering:** HTML5 Canvas 2D was chosen over WebGL for simplicity and broad compatibility, while still achieving 60fps performance.

4.3 Data Flow Diagram



Chapter 5: Technology Stack Analysis

5.1 Complete Technology Stack

Layer	Technology	Version	Purpose
Framework	Next.js	16.2.1	Server-side rendering, file-based routing, bundling
UI Library	React	19.2.4	Component-based UI architecture
Language	TypeScript	5.x	Static typing, interfaces, type safety
Styling	Tailwind CSS	4.x	Utility-first CSS framework
Styling	Custom CSS	—	Design system tokens, layout grid, animations
Canvas	HTML5 Canvas 2D API	—	Real-time simulation rendering
Audio	Web Audio API	—	Sound synthesis (bubble, sizzle, explosion)
Vision	@mediapipe/tasks-vision	0.10.33	Hand landmark detection in-browser
Backend	Python 3.x	—	Optional gesture bridge server
CV Backend	OpenCV (cv2)	—	Camera capture, frame encoding
ML Backend	MediaPipe (Python)	—	Hand detection, landmark extraction
HTTP Server	Python http.server	—	Lightweight bridge HTTP API
Build Tool	Turbopack (via Next.js)	—	Fast development builds
Linting	ESLint	9.x	Code quality enforcement
Package Manager	npm	—	Dependency management
Version Control	Git	—	Source code versioning

5.2 Why Next.js 16?

Next.js was selected over alternatives for several key reasons:

Feature	Next.js	Create React App	Vite
File-based routing	■ App Router	■ Manual	■ Manual

Server Components	■ React Server Components	■	■
SEO (SSR/SSG)	■ Built-in	■	■
API Routes	■ Built-in	■	■
TypeScript Support	■ First-class	■	■
Dev Server Speed	■ Turbopack	■■■ Webpack	■ esbuild
Metadata API	■ Built-in	■	■

Next.js 16 with the App Router provides the ideal foundation for this project by offering automatic code splitting per route, built-in metadata management for SEO, and seamless TypeScript integration.

5.3 Why React 19?

React 19 introduces several features utilized in this project:

- **forwardRef with improved performance:** The **LabSimulation** component uses **forwardRef** to expose imperative methods (**spawnObject**, **resetObjects**, **cancelReaction**) to parent components.
- **useCallback and useMemo optimization:** Extensively used to prevent unnecessary re-renders in the 60fps animation loop.
- **useImperativeHandle:** Allows the parent **Lab** page to control the simulation canvas through a clean API.

5.4 Why TypeScript?

TypeScript provides essential type safety for a complex simulation project:

```
// Example: Strongly-typed reaction product interface
export interface ReactionProduct {
  name: string;
  formula: string;
  color: string;    // CSS hex color
  emoji: string;    // Visual indicator
}

// Example: Comprehensive world object type
export interface WorldObject {
  id: string;
  type: string;
  x: number; y: number;
  vx: number; vy: number;
  rotation: number;
  angularVel: number;
  grabbed: boolean;
  chemicals?: Chemical[];
  temperature?: number;
  // ... 15+ additional typed properties
}
```

Without TypeScript, managing the complex state of 15+ properties per world object across multiple subsystems (physics, rendering, reactions, gestures) would be extremely error-prone.

Chapter 6: Frontend Architecture — Next.js & React

6.1 Application Directory Structure

The project follows Next.js 16's App Router convention:

```
frontend/
├── app/
│   ├── layout.tsx           # Root layout with metadata
│   ├── page.tsx             # Landing page (/)
│   ├── globals.css          # Design system (1,538 lines)
│   └── lab/
│       ├── page.tsx         # Main lab simulation (/lab)
│       └── dashboard/
│           ├── page.tsx     # Student dashboard (/dashboard)
│           ├── login/
│               ├── page.tsx # Authentication page (/login)
│               └── periodic-table/
│                   ├── page.tsx # Interactive periodic table (/periodic-table)
│                   ├── quiz/
│                       ├── page.tsx # Safety quiz (/quiz)
│                       └── about/
│                           ├── page.tsx # Lab guide (/about)
│                           └── components/
│                               ├── LabSimulation.tsx # Canvas simulation engine (1,984 lines)
│                               ├── LeftPanel.tsx    # Chemical selector panel (448 lines)
│                               ├── RightPanel.tsx   # Reaction monitor panel (461 lines)
│                               ├── TopBar.tsx       # Control bar component
│                               ├── BottomToolbar.tsx # Tool spawning toolbar
│                               ├── NavBar.tsx       # Global navigation bar
│                               ├── CameraView.tsx   # Camera display component
│                               ├── Toast.tsx        # Notification component
│                               └── engine/
│                                   ├── reactionEngine.ts # Pure reaction simulation engine (515 lines)
│                                   └── data/
│                                       ├── chemicals.ts # Chemical database (225 lines, 150+ chemicals)
│                                       └── reactions.ts # Reaction registry (439 lines, 21 reactions)
│                                           ├── hooks/
│                                               ├── useHandGesture.ts # MediaPipe hand tracking hook (253 lines)
│                                               ├── useCamera.ts     # Unified camera hook with bridge fallback (301 lines)
│                                               └── usePythonBridge.ts # Python bridge polling hook (118 lines)
│                                           └── audio/
│                                               ├── audioSynth.ts # Web Audio sound synthesis (174 lines)
│                                           └── package.json
│                                           └── tsconfig.json
│                                           └── next.config.ts
│                                           └── eslint.config.mjs
```

6.2 Root Layout

The root layout (**layout.tsx**) defines the global HTML structure and metadata:

```
export const metadata: Metadata = {
  title: "Virtual Chemistry Lab",
  description: "An interactive virtual chemistry laboratory where you drag tools, pour chemicals, and observe reactions in real time.",
  keywords: ["chemistry", "virtual lab", "science", "simulation", "education", "reactions"],
};
```

This ensures proper SEO indexing with descriptive title tags, meta descriptions, and keyword metadata.

6.3 Component Architecture

The project follows a **Container-Presenter pattern**:

```
Lab Page (Container)
  ■■■ TopBar (Presenter)    ■■■ Controls: Pause, Reset, Camera Toggle
  ■■■ LeftPanel (Presenter) ■■■ Chemical database browser with search
  ■■■ LabSimulation (Engine) ■■■ Canvas rendering & physics simulation
  ■■■ RightPanel (Presenter) ■■■ Reaction monitor & library
  ■■■ BottomToolbar (Presenter) ■■■ Glassware tool spawning
```

The **Lab** page (**lab/page.tsx**) orchestrates all state and passes data down via props. The **LabSimulation** component is the core engine, managing the **requestAnimationFrame** loop, physics updates, and Canvas drawing.

6.4 React Hooks Architecture

The project makes extensive use of React hooks:

Hook	File	Purpose
useState	Multiple	Component-local state management
useRef	LabSimulation.tsx	Mutable refs for animation frame, canvas, particles
useEffect	Multiple	Side effects: resize observer, camera lifecycle
useCallback	LabSimulation.tsx	Memoized animation loop, event handlers
useMemo	LeftPanel.tsx	Filtered chemical list computation
useImperativeHandle	LabSimulation.tsx	Expose imperative API to parent
forwardRef	LabSimulation.tsx	Pass ref through to functional component
useCamera (custom)	useCamera.ts	Camera + MediaPipe lifecycle management
usePythonBridge (custom)	usePythonBridge.ts	Bridge server polling

Chapter 7: TypeScript Type System & Data Modeling

7.1 Core Type Definitions

The project defines a comprehensive type system to model the chemistry domain:

Chemical Interface

```
export interface Chemical {  
  id: string;           // Unique identifier (e.g., "hcl", "naoh")  
  name: string;         // Display name (e.g., "HCl", "NaOH")  
  fullName: string;    // Full name (e.g., "Hydrochloric Acid")  
  colorName: string;   // Color description (e.g., "Pale Yellow")  
  color: string;       // CSS hex color (e.g., "#fde68a")  
  state: "Liquid" | "Solid" | "Gas"; // Physical state  
  temp: string;        // Standard temperature  
  formula: string;     // Chemical formula with subscripts  
  category: string;    // Classification category  
  emoji: string;       // Visual emoji indicator  
}
```

Reaction Record Interface

```
export interface ReactionRecord {  
  id: string;           // Unique reaction ID  
  key: string;          // Sorted reactant IDs joined with "+"  
  reactantIds: string[]; // Array of reactant chemical IDs  
  products: ReactionProduct[]; // Products generated  
  equation: string;     // Balanced chemical equation  
  type: ReactionType;   // Reaction classification  
  deltaH: number;       // Enthalpy change (kJ/mol)  
  requiresHeat: boolean; // Whether a Bunsen burner is needed  
  minTemp: number;      // Minimum temperature to trigger (°C)  
  durationSec: number;  // Base duration in seconds  
  productColor: string; // Final liquid color  
  description: string;  // Human-readable description  
}
```

7.2 Union Types for Classification

```
export type ReactionType =  
  | "neutralization" // Acid + Base → Salt + Water  
  | "precipitation"  // Double displacement → insoluble product  
  | "synthesis"      // A + B → AB  
  | "decomposition"  // AB → A + B  
  | "displacement"   // A + BC → AC + B  
  | "combustion"     // Fuel + O2 → CO2 + H2O  
  | "generic";       // Unclassified reaction  
  
export type Category = "All" | "Acid" | "Base" | "Salt" | "Organic"  
  | "Indicator" | "Oxidizer" | "Metal" | "Gas" | "Other";
```

7.3 Engine State Types

The reaction engine operates on strongly-typed state interfaces:

```

export interface SlotState {
  x: number; y: number;           // Position
  chemicals: Chemical[];         // Chemicals in container
  reactionProgress: number;       // 0-1 progress
  reacting: boolean;              // Currently reacting?
  reactionId: string | null;      // Which reaction
  liquidColor: string | null;     // Current liquid color
  liquidLevel: number;           // 0-1 fill level
  temperature: number;           // Current temperature °C
  startTime: number | null;       // Reaction start timestamp
  startTemp: number;             // Temperature at reaction start
  peakTemp: number;              // Highest temperature reached
  cancelled: boolean;            // Was reaction cancelled
  burst?: boolean;               // Did glassware explode
}

```

This comprehensive type system ensures that all chemistry simulation data flows through well-defined, compiler-verified channels.

Chapter 8: HTML5 Canvas Rendering Engine

8.1 Canvas Architecture

The simulation's visual output is rendered entirely through the **HTML5 Canvas 2D API**. A single **<canvas>** element occupies the simulation area, and all drawing—from glassware shapes to particle effects—is performed programmatically using the Canvas 2D rendering context.

Rendering Pipeline (per frame, 60fps)

1. Clear Canvas	<code>ctx.clearRect(0, 0, W, H)</code>
2. Draw Background Grid	Subtle grid lines (non-camera mode)
3. Draw Lab Floor	Gradient floor with amber glow line
4. Draw Platform Stands	3 visual stands for glassware
5. Draw Slot Liquids	Liquid fills with meniscus and reflections
6. Draw Particles	Droplets, smoke, bubbles, steam, fire, sparks, precipitate, rings
7. Draw World Objects	Vector-art glassware shapes with transformations
8. Draw Heat Shimmer	Wavy shimmer lines above heated containers
9. Draw Reaction Glow	Animated neon aura around reacting containers
10. Draw Temp Graph	Thermodynamic telemetry chart overlay

8.2 Vector Art Rendering

Instead of using pre-rendered images, all laboratory tools are drawn programmatically using Canvas path operations. This enables:

- **Resolution independence:** Tools look sharp at any canvas size
- **Dynamic state visualization:** Liquid levels, flame animation, and temperature readings update in real-time
- **Customizable skins:** Four glassware skin themes (Classic, Copper, Cyber, Gold)

Example — **Erlenmeyer Flask rendering** (simplified):

```
// Flask body outline using quadratic Bézier curves
ctx.beginPath();
ctx.moveTo(-hs * 0.25, -hs);           // Narrow neck top-left
ctx.lineTo(-hs * 0.25, -hs * 0.3);    // Neck bottom-left
ctx.lineTo(-hs * 0.9, hs * 0.7);       // Body bottom-left (widening)
ctx.quadraticCurveTo(-hs * 0.9, hs, -hs * 0.5, hs); // Bottom curve left
ctx.lineTo(hs * 0.5, hs);              // Bottom edge
ctx.quadraticCurveTo(hs * 0.9, hs, hs * 0.9, hs * 0.7); // Bottom curve right
ctx.lineTo(hs * 0.25, -hs * 0.3);     // Body right
ctx.lineTo(hs * 0.25, -hs);           // Neck top-right
ctx.closePath();
ctx.fillStyle = glassGradient;         // Glass gradient fill
ctx.fill();
ctx.strokeStyle = glassStroke;        // Glass outline
ctx.stroke();
```

8.3 Tool Types Rendered

Tool	Drawing Technique	Dynamic Elements
Flask (Erlenmeyer)	Bézier curves + ellipse rim	Liquid level, meniscus, glass shine
Beaker	Rectangular body + pour spout	Measurement lines, liquid fill
Test Tube	Narrow cylinder + rounded bottom	Liquid fill, rim ellipse
Bunsen Burner	Polygonal base + nozzle	Animated dual flame (inner blue, outer orange)
Dropper	Glass tube + rubber bulb	Drip animation, liquid inside
Graduated Cylinder	Narrow tube + graduation marks	Rim, graduation ticks
Petri Dish	Dual ellipses (top view)	3D lid edge effect
Stirring Rod	Line with rounded tip	Glass shine highlight
Spatula	Handle + curved metal blade	Metal shine reflection
Thermometer	Glass backing + mercury column	Dynamic temperature display, graduation marks
pH Meter	Probe shaft + digital display	Real-time pH reading with color-coded glow

8.4 Liquid Rendering

Liquids inside containers are rendered with multiple visual layers:

1. **Gradient Fill:** Three-stop vertical gradient from translucent to opaque
2. **Meniscus:** Elliptical curve at the liquid surface (concave meniscus simulation)
3. **Glass Reflection:** Linear gradient overlay simulating light refraction
4. **Surface Highlights:** Semi-transparent white line at the meniscus edge

8.5 Particle System

The particle system supports 8 distinct particle types:

Type	Visual	Behavior
droplet	Colored ellipse	Gravity-affected, fast decay
smoke	Radial gradient circle	Rising, lateral drift
bubble	Stroked circle + highlight	Rising inside containers, pop at surface
steam	Fading white gradient	Rising, expanding, lateral oscillation
fire	Radial gradient (blue core / orange outer)	Shrinking, fast decay

spark	Line segment (motion trail)	Gravity-affected, fast
precipitate	Small solid circle	Sinks to container bottom
ring	Expanding circle stroke	Expanding, fast fade (completion effect)

Chapter 9: Real-Time Reaction Engine

9.1 Engine Architecture

The reaction engine (**reactionEngine.ts**) is the scientific core of the application. It is designed as a **pure function** — it takes the current state, delta time, and returns the next state. This makes it:

- **Side-effect free:** No DOM manipulation, no global state mutation
- **Deterministic:** Given the same inputs, always produces the same outputs
- **Testable:** Can be unit-tested without any browser environment

```
export function tickEngine(  
  slots: SlotState[], // Current container states  
  objects: EngineObject[], // World objects (burners, etc.)  
  dt: number, // Delta time in seconds  
  now: number // Current timestamp in ms  
): EngineOutput {  
  // ... 200+ lines of simulation logic  
}
```

9.2 Tick Processing Pipeline

Each invocation of **tickEngine()** processes the following steps for every container:

```

Step 1: Thermal Dynamics
    ■■■ Calculate target temperature (185°C if heated, 25°C if not)
    ■■■ Apply exponential approach: T += (target - T) * dt * 0.5
    ■■■ Clamp to [20°C, 300°C]
    ■■■ Check overheating safety (>220°C → glassware burst)

Step 2: Liquid Color Update (non-reacting)
    ■■■ Calculate pH from chemicals
    ■■■ Check for indicator chemicals (phenolphthalein, litmus, universal)
    ■■■ Apply indicator color or blend chemical colors

Step 3: Physical Evaporation
    ■■■ If temperature > 95°C, reduce liquid level
    ■■■ If liquid depleted, clear all chemistry state

Step 4: Cancellation Detection
    ■■■ If reacting with < 2 chemicals, cancel reaction
    ■■■ Reset progress, color, and level

Step 5: Find Matching Reaction
    ■■■ Extract chemical IDs
    ■■■ Look up in reaction registry (sorted key join)
    ■■■ Try all pairwise combinations if full key fails

Step 6: Heat-Required Check
    ■■■ If reaction needs heat but no burner → "heat-required" status
    ■■■ UI displays "■ Heat Required" badge

Step 7: Trigger New Reaction
    ■■■ Check temperature ≥ minTemp
    ■■■ Check heat requirement satisfied
    ■■■ Initialize reaction state (progress=0, startTime=now)

Step 8: Advance Reaction Progress
    ■■■ Calculate speed multiplier (heated: ×2.5, stirred: ×2.0)
    ■■■ Advance progress: progress += (dt / durationSec) × speedMult
    ■■■ Calculate temperature contribution from ΔH
    ■■■ Blend liquid color: reactant mix → product color
    ■■■ Increment liquid level

Step 9: Handle Completion
    ■■■ Generate products from ReactionProduct → Chemical conversion
    ■■■ Consume reactant chemicals, keep spectators
    ■■■ Apply final indicator color or product color
    ■■■ Reset reaction state (progress=0, reacting=false)

```

9.3 Thermodynamic Modeling

The engine models enthalpy (ΔH) contribution to temperature:

```

// Scale deltaH so max contribution ≈ 80°C at ΔH = -100 kJ/mol
const tempContrib = (-reaction.deltaH / 100) * 80 * s.reactionProgress;
const reactionTemp = s.startTemp + Math.max(0, tempContrib);

```

Examples of modeled enthalpy values:

Reaction	ΔH (kJ/mol)	Effect
<chem>HCl + NaOH -> NaCl + H2O</chem>	-57.3	Moderate heat release
<chem>2H2 + O2 -> 2H2O</chem>	-483.6	Massive heat release

$\text{Ba(OH)}_2 + \text{H}_2\text{SO}_4 \rightarrow \text{BaSO}_4 \downarrow + 2\text{H}_2\text{O}$	-180.0	Significant heat release
$\text{CH}_3\text{COOH} + \text{NaOH} \rightarrow \text{CH}_3\text{COONa} + \text{H}_2\text{O}$	-55.8	Moderate heat release

9.4 pH Calculation System

The engine includes a real-time pH calculator that categorizes chemicals by acid/base strength:

```
export function calculatePH(chemicals: Chemical[]): number {
  // Strong acids: HCl, H2SO4, HNO3, HBr, HI, HClO4 → factor 3.0
  // Weak acids: other acids → factor 1.5
  // Strong bases: NaOH, KOH, Ca(OH)2, LiOH, Ba(OH)2 → factor 3.0
  // Weak bases: other bases → factor 1.5
  // pH = 7.0 + (baseFactor - acidFactor), clamped [1.0, 14.0]
}
```

9.5 Indicator Color System

Three chemical indicators dynamically change liquid color based on pH:

Indicator	Acid Color	Neutral	Base Color
Universal	Red → Orange → Yellow → Green → Cyan → Blue → Purple	Green	Purple
Phenolphthalein	Colorless	Colorless	Pink (#f472b6)
Litmus	Red	Purple	Blue

Chapter 10: Chemical Database & Reaction Registry

10.1 Chemical Database

The application contains a comprehensive chemical database with **150+ chemicals** organized into 9 categories:

Category	Count	Examples
Acid	21	HCl, H ₂ SO ₄ , HNO ₃ , CH ₃ COOH, HF, HCN
Base	16	NaOH, KOH, Ca(OH) ₂ , NH ₄ OH, Mg(OH) ₂
Salt	32	NaCl, CuSO ₄ , AgNO ₃ , NaHCO ₃ , CaCO ₃
Organic	21	Ethanol, Methanol, Acetone, Benzene, Glucose
Indicator	9	Phenolphthalein, Litmus, Universal, Methyl Orange
Oxidizer	11	KMnO ₄ , H ₂ O ₂ , K ₂ Cr ₂ O ₇ , MnO ₂ , KNO ₃
Metal	19	Zn, Fe, Cu, Mg, Al, Na, K, Au, Ag, Pt
Gas	17	CO ₂ , O ₂ , H ₂ , Cl ₂ , NH ₃ , N ₂ , He, Ar, Ne
Other	16	H ₂ O, SiO ₂ , CaO, Fe ₂ O ₃ , CuO, Sulfur, Iodine

Each chemical entry includes scientifically accurate properties:

- **Physical state** (Solid, Liquid, Gas)
- **Standard temperature**
- **Visual color** (mapped to CSS hex values)
- **Chemical formula** (with proper subscript notation: H₂SO₄)
- **Category classification**

10.2 Reaction Registry

The reaction registry defines **21 chemical reactions** with full thermodynamic data:

#	Reaction	Type	ΔH (kJ/mol)	Heat Required
1	HCl + NaOH → NaCl + H ₂ O	Neutralization	-57.3	No
2	2H ₂ + O ₂ → 2H ₂ O	Synthesis	-483.6	Yes
3	AgNO ₃ + NaCl → AgCl↓ + NaNO ₃	Precipitation	-65.5	No

4	$\text{CuSO}_4 + 2\text{NaOH} \rightarrow \text{Cu(OH)}_2 \downarrow + \text{Na}_2\text{SO}_4$	Precipitation	-45.2	No
5	$\text{H}_2\text{SO}_4 + 2\text{NaOH} \rightarrow \text{Na}_2\text{SO}_4 + 2\text{H}_2\text{O}$	Neutralization	-112.4	No
6	$\text{CH}_3\text{COOH} + \text{NaOH} \rightarrow \text{CH}_3\text{COONa} + \text{H}_2\text{O}$	Neutralization	-55.8	No
7	$\text{HNO}_3 + \text{KOH} \rightarrow \text{KNO}_3 + \text{H}_2\text{O}$	Neutralization	-57.0	No
8	$\text{Ba(OH)}_2 + \text{H}_2\text{SO}_4 \rightarrow \text{BaSO}_4 \downarrow + 2\text{H}_2\text{O}$	Precipitation	-180.0	No
9	$\text{HCl} + \text{KOH} \rightarrow \text{KCl} + \text{H}_2\text{O}$	Neutralization	-57.3	No
10	$\text{FeSO}_4 + 2\text{NaOH} \rightarrow \text{Fe(OH)}_2 \downarrow + \text{Na}_2\text{SO}_4$	Precipitation	-40.0	No
11	$2\text{HCl} + \text{Mg(OH)}_2 \rightarrow \text{MgCl}_2 + 2\text{H}_2\text{O}$	Neutralization	-109.6	No
12	$\text{AgNO}_3 + \text{KCl} \rightarrow \text{AgCl} \downarrow + \text{KNO}_3$	Precipitation	-65.5	No
13	$\text{CH}_3\text{COOH} + \text{KOH} \rightarrow \text{CH}_3\text{COOK} + \text{H}_2\text{O}$	Neutralization	-55.8	No
14	$\text{HCl} + \text{LiOH} \rightarrow \text{LiCl} + \text{H}_2\text{O}$	Neutralization	-57.3	No
15	$\text{H}_2\text{CO}_3 + 2\text{NaOH} \rightarrow \text{Na}_2\text{CO}_3 + 2\text{H}_2\text{O}$	Neutralization	-47.1	No
16	$\text{HNO}_3 + \text{NaOH} \rightarrow \text{NaNO}_3 + \text{H}_2\text{O}$	Neutralization	-57.0	No
17	$\text{HF} + \text{NaOH} \rightarrow \text{NaF} + \text{H}_2\text{O}$	Neutralization	-68.6	No
18	$\text{H}_3\text{PO}_4 + 3\text{NaOH} \rightarrow \text{Na}_3\text{PO}_4 + 3\text{H}_2\text{O}$	Neutralization	-75.4	No
19	$\text{HBr} + \text{NaOH} \rightarrow \text{NaBr} + \text{H}_2\text{O}$	Neutralization	-57.3	No
20	$\text{Ca(OH)}_2 + 2\text{HCl} \rightarrow \text{CaCl}_2 + 2\text{H}_2\text{O}$	Neutralization	-115.4	No
21	$\text{HCl} + \text{H}_2\text{SO}_4 \rightarrow \text{ClSO}_3\text{H (fuming)}$	Generic	-15.0	No

10.3 Reaction Lookup Algorithm

The lookup uses a sorted-key strategy:

```
export function findReaction(chemicalIds: string[]): ReactionRecord | null {
  // 1. Filter out water (solvent/spectator)
  const activeIds = chemicalIds.filter((id) => id !== "h2o");

  // 2. Try full key match (all chemicals sorted + joined)
  const fullKey = [...activeIds].sort().join("+");
  if (REACTION_REGISTRY[fullKey]) return REACTION_REGISTRY[fullKey];

  // 3. Fallback: try all pairwise combinations
  for (let i = 0; i < activeIds.length; i++) {
    for (let j = i + 1; j < activeIds.length; j++) {
      const pairKey = [activeIds[i], activeIds[j]].sort().join("+");
      if (REACTION_REGISTRY[pairKey]) return REACTION_REGISTRY[pairKey];
    }
  }
  return null;
}
```

This two-phase lookup ensures that reactions are found regardless of the order in which chemicals are added, and that multi-component mixtures can still trigger pairwise reactions.

Chapter 11: Physics Simulation System

11.1 Physics Constants

The simulation uses the following physics parameters:

Constant	Value	Purpose
GRAVITY	600 px/s ²	Gravitational acceleration
DAMPING	0.65	Coefficient of restitution (bounce)
ANGULAR_DAMPING	0.85	Rotational velocity decay
OBJECT_SIZE	72 px	Standard object bounding size
FLOOR_PAD	140 px	Floor position from bottom

11.2 Physics Update Loop

Every animation frame, the following physics computations are performed for each non-grabbed object:

```
// 1. Apply gravity
obj.vy += GRAVITY * dt;

// 2. Update position based on velocity
obj.x += obj.vx * dt;
obj.y += obj.vy * dt;

// 3. Update rotation
obj.rotation += obj.angularVel * dt;
obj.angularVel *= ANGULAR_DAMPING;

// 4. Floor collision
if (obj.y > floorY - OBJECT_SIZE / 2) {
  obj.y = floorY - OBJECT_SIZE / 2; // Clamp to floor
  obj.vy *= -DAMPING;               // Bounce with energy loss
  obj.vx *= 0.85;                   // Friction
  if (Math.abs(obj.vy) < 20) obj.vy = 0; // Rest threshold
}

// 5. Wall boundaries
if (obj.x < OBJECT_SIZE / 2) { obj.x = OBJECT_SIZE / 2; obj.vx *= -0.5; }
if (obj.x > W - OBJECT_SIZE / 2) { obj.x = W - OBJECT_SIZE / 2; obj.vx *= -0.5; }
```

11.3 Drag-and-Drop Physics

When objects are grabbed (via mouse or gesture), velocity is computed from position deltas:

```
// Apply velocity based on movement speed (for natural throw physics)
obj.vx = (newX - obj.x) * 15;
obj.vy = (newY - obj.y) * 15;
obj.x = newX;
obj.y = newY;
```

When released, the accumulated velocity creates a natural "throw" effect with continued momentum, gravity, and eventual settling.

11.4 Dropper-Container Proximity Detection

The system detects when a dropper is positioned over a container:

```
const dist = Math.hypot(dropper.x - container.x, (dropper.y + OBJECT_SIZE / 2) - container.y);
if (dist < OBJECT_SIZE * 0.9) {
  // Transfer chemical from dropper to container
  container.chemicals = [...container.Chems, dropper.chemical];
  container.liquidLevel = Math.min(1, container.liquidLevel + 0.35);
}
```

Chapter 12: Hand Gesture Recognition — MediaPipe Integration

12.1 MediaPipe Hand Landmarker

The project integrates Google's **MediaPipe Hand Landmarker** model to detect hand landmarks directly in the browser. The model:

- Detects **21 3D landmarks** per hand (wrist, finger joints, fingertips)
- Supports **up to 2 hands simultaneously** (dual-hand interaction)
- Runs at **30+ FPS** via WebAssembly/WebGPU acceleration
- Uses the **float16** precision model for optimal browser performance

12.2 Implementation Architecture

The gesture system uses a **dual-hook architecture**:

1. **useCamera hook** — Primary camera hook with automatic bridge-to-browser fallback:

Toggle Camera → Try Python Bridge → If unavailable → Fall back to Browser Camera + MediaPipe

2. **usePythonBridge hook** — Direct bridge polling for Python-based hand tracking:

Poll /state every 75ms → Parse hand data → Update bridgeFrameUrl

12.3 MediaPipe Configuration

```
const lm = await HandLandmarker.createFromOptions(vision, {
  baseOptions: {
    modelAssetPath: "hand_landmarker.task",
    delegate: "GPU", // WebGPU acceleration
  },
  runningMode: "VIDEO",
  numHands: 2, // Dual-hand support
  minHandDetectionConfidence: 0.6, // Detection threshold
  minHandPresenceConfidence: 0.5, // Presence threshold
  minTrackingConfidence: 0.5, // Tracking threshold
});
```

12.4 Pinch Gesture Detection

A pinch gesture is detected by measuring the Euclidean distance between the thumb tip (landmark 4) and index finger tip (landmark 8):

```
const PINCH_THRESHOLD = 0.07; // Normalized distance (0-1)

const dx = indexTip.x - thumbTip.x;
const dy = indexTip.y - thumbTip.y;
const dist = Math.sqrt(dx * dx + dy * dy);
const isPinching = dist < PINCH_THRESHOLD;
```

12.5 Hand Data Interface

```
export interface HandData {
  label: "Left" | "Right";
  wrist: { x: number; y: number }; // Normalized 0-1
  indexTip: { x: number; y: number };
  thumbTip: { x: number; y: number };
  pinching: boolean; // Pinch state
  landmarks: { x: number; y: number; z: number }[]; // All 21 landmarks
}
```

12.6 Gesture-to-Action Mapping

Gesture	Action
Pinch (single hand)	Grab nearest object
Move while pinching	Drag object to new position
Release pinch	Drop object (with physics momentum)
Pinch with second hand (while holding dropper)	Dispense chemical drop
Pinch with second hand (while holding burner)	Toggle flame on/off

12.7 Hand Skeleton Overlay

When the camera is active, a real-time hand skeleton is drawn on the overlay canvas:

```
const CONNECTIONS = [
  [0,1],[1,2],[2,3],[3,4], // Thumb chain
  [0,5],[5,6],[6,7],[7,8], // Index finger chain
  [0,9],[9,10],[10,11],[11,12], // Middle finger chain
  [0,13],[13,14],[14,15],[15,16], // Ring finger chain
  [0,17],[17,18],[18,19],[19,20], // Pinky finger chain
  [5,9],[9,13],[13,17], // Palm connections
];
```

The skeleton renders in **teal** when the hand is open and switches to **amber** when pinching.

12.8 Performance Optimization

- **State update throttling:** React state updates are throttled to ~10fps (every 100ms) while the actual detection runs at full camera frame rate. The **handDataRef** mutable ref provides the latest data to the animation loop without triggering re-renders.
- **Mirrored coordinates:** The camera feed is horizontally mirrored (**transform: scaleX(-1)**) and landmark x-coordinates are inverted (**1 - pts[a].x**) for natural interaction.

Chapter 13: Audio Snap Detection & Web Audio API

13.1 Web Audio API Overview

The project uses the **W3C Web Audio API** for two purposes:

1. **Sound Synthesis** (output) — Programmatic generation of laboratory sounds
2. **Snap Detection** (input, via Python backend) — Real-time acoustic event detection

13.2 AudioSynth Class Architecture

The **AudioSynth** class (**audioSynth.ts**) implements four distinct sound effects using oscillator nodes, gain nodes, and filter nodes:

Bubble Pop Sound

```
playBubble() {
  const osc = ctx.createOscillator();
  osc.type = "sine";
  osc.frequency.setValueAtTime(200 + Math.random() * 150, now);
  osc.frequency.exponentialRampToValueAtTime(endFreq, now + 0.12);
  // Quick pitch sweep: rising sine wave simulating bubble pop
}
```

Sizzle Sound (Continuous)

Uses a **looping white noise buffer** filtered through a **bandpass filter** at 1600Hz:

```
playSizzle(intensity: number) {
  const source = ctx.createBufferSource();
  source.buffer = noiseBuffer; // 2-second white noise
  source.loop = true;

  const filter = ctx.createBiquadFilter();
  filter.type = "bandpass";
  filter.frequency.setValueAtTime(1600, now);

  const gain = ctx.createGain();
  gain.gain.setValueAtTime(intensity * 0.15, now);
  // Creates crackling sizzle proportional to temperature
}
```

Reaction Success Chime

Plays a **C-major chord arpeggio** (C5→E5→G5→C6) using staggered sine oscillators:

```
const freqs = [523.25, 659.25, 783.99, 1046.50]; // C5, E5, G5, C6
freqs.forEach((freq, idx) => {
  osc.frequency.setValueAtTime(freq, now + idx * 0.08);
  // Each note 80ms apart, creating ascending arpeggio
});
```

Explosion Sound

Combines **low-pass filtered noise** with a **sub-bass oscillator** (80→20Hz):

```
// Low-pass noise blast
filter.frequency.setValueAtTime(450, now);
filter.frequency.exponentialRampToValueAtTime(10, now + 0.6);

// Sub-bass rumble
subOsc.frequency.setValueAtTime(80, now);
subOsc.frequency.linearRampToValueAtTime(20, now + 0.5);
```

13.3 Audio Context Lazy Initialization

Due to browser autoplay policies, the **AudioContext** is lazily initialized on the first user interaction:

```
private initCtx() {
  if (this.ctx) return;
  const AudioContextClass = window.AudioContext || window.webkitAudioContext;
  if (AudioContextClass) {
    this.ctx = new AudioContextClass();
    this.createNoiseBuffer();
  }
}
```

13.4 Snap Detection (Python Backend)

The Python backend includes an **audio_system.py** module that:

1. Captures microphone input using the system's audio API
2. Applies bandpass filtering to isolate snap frequency range
3. Detects amplitude spikes matching snap characteristics
4. Distinguishes between single snaps and double snaps (within a timing window)
5. Publishes events via the bridge: **"SINGLE_SNAP"** or **"DOUBLE_SNAP"**

Actions triggered by snaps:

Event	Action
SINGLE_SNAP	Turn Bunsen burner flame ON
DOUBLE_SNAP	Turn Bunsen burner flame OFF

Chapter 14: Python Backend Bridge Server

14.1 Bridge Architecture

The Python backend serves as an optional bridge between native system hardware (webcam, microphone) and the browser-based frontend. It runs as a lightweight HTTP server on **http://127.0.0.1:8765**.

14.2 Server Implementation

The bridge uses Python's built-in **http.server** module with **ThreadingHTTPServer** for concurrent request handling:

```
class _BridgeHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        route = self.path.split("?", 1)[0]

        if route == "/health":
            return self._write_json(200, {"ok": True})

        if route == "/state":
            # Returns JSON: {hands: [...], audioEvent, paused, timestamp}
            return self._write_json(200, snapshot)

        if route == "/frame.jpg":
            # Returns JPEG-encoded camera frame
            return self._write_bytes(200, frame_bytes, "image/jpeg")
```

14.3 API Endpoints

Endpoint	Method	Response	Purpose
/health	GET	{"ok": true}	Server health check
/state	GET	{"hands": [...], "audioEvent": "...", "paused": false}	Hand data + audio events
/frame.jpg	GET	Binary JPEG image	Camera frame (with no-cache headers)

14.4 CORS Configuration

The bridge enables cross-origin requests from the Next.js dev server:

```
def end_headers(self):
    self.send_header("Access-Control-Allow-Origin", "*")
    self.send_header("Access-Control-Allow-Methods", "GET, OPTIONS")
    self.send_header("Access-Control-Allow-Headers", "Content-Type")
    super().end_headers()
```

14.5 Frame Publishing

Camera frames are encoded as JPEG with configurable quality and published via a thread-safe lock:

```
def publish_bridge_frame(frame_bgr, quality=70):
    ok, encoded = cv2.imencode(".jpg", frame_bgr,
                               [int(cv2.IMWRITE_JPEG_QUALITY), int(quality)])
    with _state_lock:
        _frame_bytes = encoded.tobytes()
        _state["frameTimestamp"] = time.time()
```

14.6 Bridge Fallback Mechanism

The **useCamera** hook implements an automatic fallback:

1. User toggles camera ON
2. Try Python Bridge mode → Poll /state endpoint
3. If bridge unavailable (connection refused) →
4. Auto-switch to Browser Camera mode
5. Initialize MediaPipe in-browser via dynamic import
6. Start detection loop with requestAnimationFrame

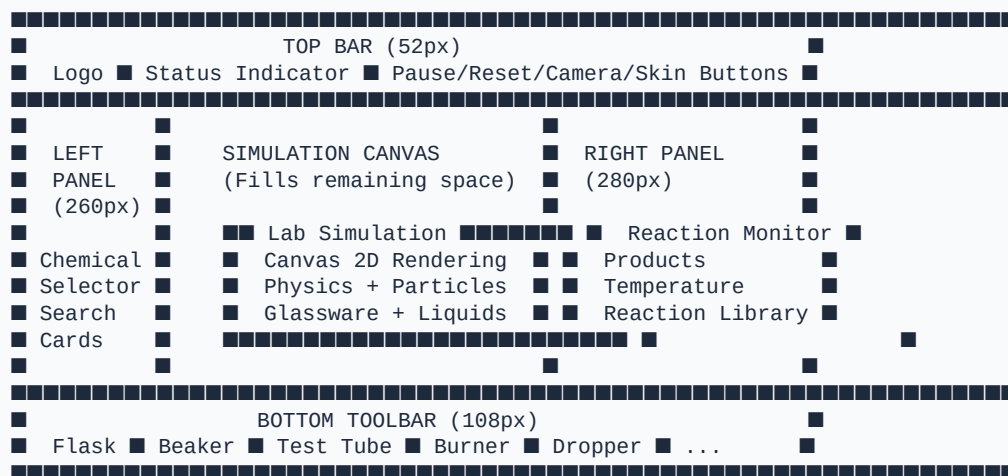
This ensures the application works both with and without the Python backend running.

Chapter 15: UI/UX Design & Component Architecture

15.1 Lab Page Layout

The main lab page uses a **CSS Grid layout** with five distinct regions:

```
.lab-shell {  
  display: grid;  
  grid-template-areas:  
    "topbar topbar topbar"  
    "leftpanel sim rightpanel"  
    "bottombar bottombar bottombar";  
  grid-template-rows: 52px 1fr 108px;  
  grid-template-columns: 260px 1fr 280px;  
  height: 100vh;  
  width: 100vw;  
  overflow: hidden;  
}
```



15.2 Component Details

Left Panel — Chemical Selector

- **Search bar** with real-time filtering across name, formula, category, and ID
- **Category filter tabs** — 10 pill-shaped buttons for Acid, Base, Salt, Organic, etc.
- **Result counter** — Shows "X of 150+ chemicals" dynamically
- **Chemical cards** — Each card displays:
 - Color swatch (circular, matching chemical color)
 - Name (formula notation)
 - Full name
 - State, Temperature, Color, Formula (2×2 info grid)
 - Category badge
 - "Add" button to spawn dropper
- **Chemical Info Modal** — Click any card to see:
 - NFPA 704 diamond (rendered as inline SVG)
 - Physical properties grid

- Hazard statements
- "Add to Lab" action button

Right Panel — Reaction Monitor

- **Active reaction card** with status badge (Idle / Heat Required / Reacting / Complete)
- **Chemical equation display** (reactants in gold, arrow, products in green)
- **Progress bar** (gradient from rose → amber → emerald)
- **Temperature gauge** (start/current/peak with color-coded bar)
- **Duration timer** (formatted as ms or seconds)
- **Cancel button** (during active reactions)
- **Product list** (color swatches + formula + name)
- **Last completed card** (4 stats: Reactants, Duration, Peak Temp, Type)
- **Download Lab Report** button (generates .md file)
- **Reaction Library** — Scrollable list of all 21 reactions with type badges, heat indicators, and ΔH values

Top Bar

- Logo with amber gradient glow
- Simulation status indicator (green pulsing dot = running, amber = paused)
- Control buttons: Pause/Resume, Clear All, Toggle Camera, Glassware Skin selector

Bottom Toolbar

- Tool spawning buttons for: Flask, Beaker, Test Tube, Bunsen Burner, Dropper, Graduated Cylinder, Petri Dish, Stirring Rod, Spatula, Thermometer, pH Meter

15.3 Toast Notification System

The **Toast** component displays temporary notifications:

```
interface ToastProps {
  icon: string;    // Emoji icon
  text: string;    // Notification text
}
```

Used for events like: "■■■ Glassware burst due to excessive heat!"

Chapter 16: CSS Design System & Theming

16.1 Design Token System

The project implements a comprehensive CSS custom properties (variables) design system:

```
:root {  
  /* Color Palette */  
  --bg-void: #050810;      /* Deepest background */  
  --bg-deep: #080c18;      /* Deep background */  
  --bg-panel: #0d1220;      /* Panel background */  
  --bg-card: #111827;       /* Card background */  
  --amber: #f59e0b;         /* Primary accent */  
  --teal: #14b8a6;          /* Secondary accent */  
  --violet: #8b5cf6;        /* Tertiary accent */  
  --rose: #f43f5e;          /* Danger/alert */  
  --emerald: #10b981;       /* Success */  
  
  /* Typography */  
  --text-primary: #f1f5f9;  
  --text-secondary: #94a3b8;  
  --text-muted: #475569;  
  
  /* Glassmorphism */  
  --glass-bg: rgba(13, 18, 32, 0.85);  
  --glass-border: rgba(255, 255, 255, 0.08);  
  
  /* Shadows */  
  --shadow-amber: 0 0 20px rgba(245, 158, 11, 0.2), 0 0 40px rgba(245, 158, 11, 0.08);  
}
```

16.2 Visual Design Philosophy

The application follows a **dark-mode-first, sci-fi laboratory aesthetic**:

- **Deep void backgrounds** (#050810) create a sense of depth
- **Amber accent** (#f59e0b) provides warm contrast and draws attention to interactive elements
- **Glassmorphism panels** with backdrop-filter blur and subtle borders
- **Gradient accents** using CSS **linear-gradient** and **radial-gradient** for depth
- **Glow effects** via **box-shadow** with colored rgba values

16.3 CSS File Statistics

The **globals.css** file spans **1,538 lines** and defines:

- 52+ CSS custom properties
- Grid layout system for the lab shell
- Button variants (amber, ghost, danger, pause)
- Chemical card components
- Simulation area styling
- Panel components (left, right)
- Hand gesture indicators
- Animation keyframes
- Responsive utilities
- Custom scrollbar styling

- Landing page styles
- Dashboard page styles
- Login page styles
- Periodic table grid
- Quiz page styles

16.4 Key Animations

```
@keyframes pulse-dot {  
  0%, 100% { opacity: 1; }  
  50% { opacity: 0.4; }  
}  
  
@keyframes reactionPulse {  
  from { box-shadow: 0 0 8px rgba(244, 63, 94, 0.15); }  
  to   { box-shadow: 0 0 18px rgba(244, 63, 94, 0.35); }  
}  
  
@keyframes flickerFlame {  
  from { transform: translateX(-50%) scale(1); }  
  to   { transform: translateX(-50%) scale(-1); }  
}
```

Chapter 17: State Management Architecture

17.1 State Management Strategy

The project uses **React's built-in state management** (useState + useRef + useCallback) without external libraries like Redux or Zustand. This decision is justified by:

- The simulation state needs **frame-level access** (60fps), which is better served by mutable refs than reactive state
- The component tree is **relatively shallow** (max 3 levels deep)
- **Cross-component communication** is limited to parent-child prop passing

17.2 State Categories

Category	Storage	Mechanism	Update Frequency
World objects	objectsRef (mutable ref)	Direct mutation in animation loop	60fps
Particles	particlesRef (mutable ref)	Array filter/push in loop	60fps
Reaction report	useState in Lab page	Callback from engine tick	Per-tick
Hand data	handDataRef (mutable ref)	MediaPipe detection loop	Camera FPS
Hand data (UI)	useState in hook	Throttled to 10fps	10fps
UI state (paused)	useState in Lab page	User action	On demand
Canvas dimensions	sizeRef (mutable ref)	ResizeObserver	On resize
User session	localStorage	JSON serialization	On login/logout
Chemical search	useState in LeftPanel	Input change	On keystroke

17.3 The Ref-vs-State Decision

The key architectural decision is the use of **mutable refs** for high-frequency data:

```
const objectsRef = useRef<WorldObject[]>(objects);    // 60fps mutation
const particlesRef = useRef<Particle[]>([]);          // 60fps mutation
const handDataRef = useRef<HandData[]>([]);           // Camera FPS

// React state only used for data that triggers UI re-renders:
const [handData, setHandData] = useState<HandData[]>([]); // 10fps throttled
```

This hybrid approach ensures the animation loop runs at full speed without being blocked by React's reconciliation cycle.

17.4 User Session Persistence

User authentication uses **localStorage** for session persistence:

```
// Login
localStorage.setItem("lab_user", JSON.stringify({ name, exp: 0, role: "Scholar" }));

// Check authentication
const user = localStorage.getItem("lab_user");
if (!user) router.push("/login?redirect=/lab");

// Logout
localStorage.removeItem("lab_user");
```

Chapter 18: Application Routing & Page Structure

18.1 Route Map

Route	Page Component	Authentication	Description
/	LandingPage	Public	Hero section, features grid, CTAs
/lab	Lab (main simulation)	Protected	Full laboratory simulation
/dashboard	Dashboard	Protected	Student progress tracking
/login	LoginPage	Public	Simulated authentication
/periodic-table	PeriodicTable	Public	Interactive periodic table
/quiz	Quiz	Public	Chemistry safety quiz
/about	About	Public	Lab guide and documentation

18.2 Next.js App Router

The project uses **Next.js App Router** (introduced in Next.js 13+):

```
app/  
■■■ layout.tsx      # Root layout (wraps all pages)  
■■■ page.tsx        # Home page (/)  
■■■ lab/page.tsx    # Lab page (/lab)  
■■■ dashboard/page.tsx # Dashboard (/dashboard)  
■■■ login/page.tsx  # Login (/login)  
■■■ periodic-table/page.tsx # Periodic table  
■■■ quiz/page.tsx   # Quiz  
■■■ about/page.tsx  # About
```

Each page is a **Client Component** ("**use client**" directive) because they all require browser APIs (Canvas, MediaPipe, localStorage, etc.).

18.3 Navigation Component

The **Navbar** component provides a persistent header across all pages:

- **Brand logo** with amber gradient and flask emoji
- **Navigation links:** Home, Dashboard, Periodic Table, Safety Quiz, Lab Guide
- **Launch Lab button** — Primary CTA with amber glow
- **User info** — Displays name and EXP when logged in
- **Logout button** — Red-tinted danger action

18.4 Protected Route Pattern

Routes requiring authentication use a client-side redirect pattern:

```
onClick={() => {
  if (localStorage.getItem("lab_user")) {
    router.push("/lab");
  } else {
    router.push("/login?redirect=/lab");
  }
}}
```

After successful login, the user is redirected back to their intended destination via the **redirect** query parameter.

18.5 Landing Page Features

The landing page highlights four key features:

Feature	Icon	Description
Real-Time Solver	■ ■	Thermodynamics, liquid spills, gas releases, color shifts
Hand Gesture Control	■ ■	MediaPipe hand tracking, pinch-to-grab interaction
Audio Snap Ignition	■	Real-time snap detection for Bunsen burner control
150+ Chemical Database	■	Comprehensive acid, base, salt, organic, metal database

Chapter 19: Performance Optimization Techniques

19.1 Rendering Optimizations

Technique	Implementation	Impact
<code>requestAnimationFrame</code>	Single animation loop at native refresh rate	Smooth 60fps
Delta-time capping	<code>Math.min(dt, 0.05)</code> prevents physics explosions	Stability
Direct Canvas mutation	No React re-renders during frame drawing	Zero overhead
Particle culling	Particles with <code>life <= 0</code> are filtered each frame	Memory efficiency
ResizeObserver	Canvas dimensions cached in ref, updated only on resize	No per-frame DOM queries

19.2 React Rendering Optimizations

Technique	Usage	Purpose
<code>useCallback</code>	Animation loop, event handlers	Prevent function recreation
<code>useMemo</code>	Chemical list filtering	Avoid recalculation on every render
<code>useRef</code> for mutable state	Objects, particles, hand data	Avoid triggering re-renders
Throttled state updates	Hand data UI updates at 10fps	Reduce React reconciliation
<code>forwardRef</code> + <code>useImperativeHandle</code>	Lab simulation API	Clean parent-child communication

19.3 MediaPipe Optimizations

- **GPU delegate:** `delegate: "GPU"` enables WebGPU/WebGL acceleration
- **Float16 model:** Half-precision model reduces memory and computation
- **Lazy loading:** MediaPipe WASM modules are loaded on first camera activation via dynamic `import()`
- **Detection throttling:** Full landmark data flows through refs; React state updates are throttled

19.4 Network Optimizations

- **Bridge polling interval:** 75ms (13fps) — sufficient for gesture smoothness while minimizing CPU usage
- **JPEG quality:** 70% compression for bridge frame transmission
- **No-cache headers:** Frame.jpg responses include `Cache-Control: no-store` to prevent stale frames
- **CDN delivery:** MediaPipe WASM/model assets served from jsDelivr CDN

19.5 Memory Management

- **Particle pool limiting:** Maximum particle count managed through life-based filtering
- **Graph history:** Temperature graph limited to 80 data points (`shift()` on overflow)

- **Camera cleanup:** Streams, animation frames, and MediaPipe instances properly disposed on unmount
-

Chapter 20: Security & Data Handling

20.1 Client-Side Security

Concern	Mitigation
XSS (Cross-Site Scripting)	React's JSX auto-escapes all rendered values
Camera access	Browser permission prompt via <code>getUserMedia</code>
localStorage tampering	Non-sensitive data only (name, EXP score)
CORS for bridge	Origin: * (localhost only, development mode)

20.2 Data Privacy

- **No server-side data storage:** All user data is stored exclusively in `localStorage`
- **No telemetry or analytics:** The application does not track or transmit user behavior
- **Camera feed processing:** Hand detection runs entirely client-side; no frames are transmitted to external servers (unless the Python bridge is running locally)
- **Audio processing:** Snap detection (via Python) processes audio locally; no recordings are stored

20.3 Content Security

- **External dependencies** are loaded from trusted CDNs:
- `cdn.jsdelivr.net` — MediaPipe WASM runtime
- `storage.googleapis.com` — MediaPipe hand landmarker model
- **No dynamic code execution:** No `eval()`, `innerHTML`, or `dangerouslySetInnerHTML` usage
- **TypeScript strict mode:** Compiler catches type errors at build time

20.4 Input Validation

- **Chemical search:** Input is sanitized through string comparison (no regex injection risk)
- **Chemical IDs:** Reaction lookup uses hardcoded string keys; no user-provided code is executed
- **Bridge responses:** JSON parsing is wrapped in error handling to prevent crashes from malformed data

Chapter 21: Testing & Quality Assurance

21.1 Code Quality Enforcement

Tool	Configuration	Purpose
TypeScript (strict mode)	<code>tsconfig.json</code> — <code>"strict": true</code>	Compile-time type checking
ESLint	<code>eslint.config.mjs</code> — <code>eslint-config-next</code>	Code style and best practices
Next.js Build	<code>next build</code>	Production build validation

21.2 TypeScript Strict Mode Configuration

```
{
  "compilerOptions": {
    "target": "ES2017",
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "moduleResolution": "bundler",
    "isolatedModules": true,
    "jsx": "react-jsx"
  }
}
```

Strict mode enables:

- **strictNullChecks** — No implicit null/undefined
- **strictFunctionTypes** — Correct function parameter variance
- **noImplicitAny** — All values must have explicit types

21.3 Testing Strategy

Test Type	Approach	Scope
Unit Testing	Pure function testing of <code>tickEngine()</code> , <code>findReaction()</code> , <code>calculatePH()</code>	Reaction engine logic
Component Testing	React Testing Library for LeftPanel, RightPanel	UI rendering
Integration Testing	Full lab page with mocked Canvas context	End-to-end simulation
Manual Testing	Browser testing with multiple chemical combinations	Visual verification

21.4 Reaction Engine Testability

The reaction engine is designed for pure-function testing:

```
// Example unit test for pH calculation
const acidChems = [{ id: "hcl", category: "Acid", ... }];
const pH = calculatePH(acidChems);
assert(pH >= 1.0 && pH <= 6.0); // Strong acid → low pH

// Example unit test for reaction lookup
const reaction = findReaction(["hcl", "naoh"]);
assert(reaction.id === "hcl_naoh");
assert(reaction.type === "neutralization");
assert(reaction.deltaH === -57.3);
```

21.5 Browser Compatibility Testing

Browser	Version	Status
Google Chrome	125+	■ Full support
Mozilla Firefox	130+	■ Full support
Microsoft Edge	125+	■ Full support
Safari	18+	■■ WebGPU support varies

Chapter 22: Conclusion & Future Scope

22.1 Summary of Achievements

The Virtual Chemistry Laboratory successfully demonstrates the capabilities of modern web technologies for science education:

1. **Real-time chemical reaction simulation** with 21 reactions, proper stoichiometry, and thermodynamic modeling (enthalpy ΔH calculations)
2. **Physics-driven object interaction** with gravity, collision, angular momentum, and natural throw physics on HTML5 Canvas
3. **Computer vision gesture control** using Google MediaPipe Hand Landmarker with dual-hand support and pinch gesture detection
4. **Acoustic snap detection** for hands-free Bunsen burner control
5. **Comprehensive chemical database** of 150+ chemicals with accurate physical and chemical properties
6. **Immersive audio feedback** using the Web Audio API with synthesized bubble, sizzle, explosion, and success sounds
7. **Premium dark-mode UI** with glassmorphism design, CSS Grid layout, and micro-animations
8. **Cross-platform delivery** as a standard web application requiring no installation

22.2 Key Web Technologies Utilized

The project showcases the following web technologies and APIs:

- **Next.js 16** — Server-side rendering, file-based routing, metadata API
- **React 19** — Component architecture, hooks (useState, useRef, useCallback, useMemo, useImperativeHandle, forwardRef)
- **TypeScript 5** — Static typing, interfaces, union types, generics
- **HTML5 Canvas 2D API** — Programmatic vector rendering at 60fps
- **Web Audio API** — Real-time audio synthesis with oscillators, filters, and gain nodes
- **MediaPipe Tasks Vision** — In-browser ML inference for hand landmark detection
- **MediaStream API** — Webcam access via `getUserMedia()`
- **ResizeObserver API** — Responsive canvas sizing
- **requestAnimationFrame** — High-performance animation loop
- **localStorage API** — Client-side session persistence
- **Fetch API** — Bridge server HTTP polling
- **Blob API** — Lab report file download generation
- **CSS Custom Properties** — Theming and design token system
- **CSS Grid & Flexbox** — Complex layout composition

22.3 Limitations

1. **No persistent backend:** User progress is lost if `localStorage` is cleared
2. **Limited to 2D rendering:** 3D visualization would improve immersion
3. **Reaction database scope:** 21 reactions cover common types but do not include electrochemistry, organic synthesis, or biochemistry
4. **Mobile responsiveness:** The lab layout is optimized for desktop (1280px+)

5. **No real-time multiplayer:** Students cannot collaborate in a shared virtual lab

22.4 Future Scope

Enhancement	Technology	Impact
3D Rendering	Three.js / React Three Fiber	Photorealistic glassware and liquids
Voice Commands	Web Speech API	"Pour HCl into the beaker"
AI Lab Assistant	Gemini API / LLM integration	Real-time chemistry Q&A during experiments
Multiplayer Labs	WebSocket / WebRTC	Collaborative experiments
Persistent Backend	Firebase / Supabase	Cloud-saved progress and experiment history
AR Integration	WebXR API	Overlay virtual chemicals on physical lab bench
Expanded Reactions	Extended reaction registry	100+ reactions including organic and biochemistry
Assessment Analytics	Dashboard enhancement	Learning analytics and performance tracking
Mobile App	React Native / PWA	Full mobile laboratory experience
Accessibility	WCAG 2.1 compliance	Screen reader support, keyboard navigation

22.5 Final Remarks

This project demonstrates that modern web technologies have matured to a point where complex, real-time scientific simulations — including computer vision, audio processing, physics engines, and chemistry modeling — can all run directly in the browser at 60fps. The combination of Next.js, React, TypeScript, HTML5 Canvas, MediaPipe, and the Web Audio API creates a powerful platform that rivals native desktop applications while maintaining the universal accessibility of the web.

The Virtual Chemistry Laboratory is not just a demonstration of technical capability; it is a practical educational tool that can bring safe, free, and repeatable chemistry experimentation to students worldwide, regardless of their physical laboratory access.

References

1. Dalgarno, B., Bishop, A.G., Adlong, W., & Bedgood Jr, D.R. (2009). "Effectiveness of a Virtual Laboratory as a preparatory resource for Distance Education chemistry students." *Computers & Education*, 53(3), 853-865.
2. Lugaresi, C., et al. (2019). "MediaPipe: A Framework for Building Perception Pipelines." *arXiv preprint arXiv:1906.08172*.
3. Roberts, C., Wakefield, G., & Wright, M. (2014). "The Web Browser as Synthesizer and Interface." *Proceedings of NIME*.

4. Meta Open Source. (2024). "React 19 Documentation." <https://react.dev/>
 5. Vercel Inc. (2026). "Next.js 16 Documentation." <https://nextjs.org/docs>
 6. Google LLC. (2024). "MediaPipe Hand Landmarker."
https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
 7. W3C. (2021). "Web Audio API Specification." <https://www.w3.org/TR/webaudio/>
 8. W3C. (2023). "HTML Canvas 2D Context Specification." <https://html.spec.whatwg.org/multipage/canvas.html>
 9. ECMA International. (2023). "TypeScript Language Specification." <https://www.typescriptlang.org/docs/>
 10. MDN Web Docs. (2024). "MediaStream API."
https://developer.mozilla.org/en-US/docs/Web/API/MediaStream_API
-

End of Report

Total Pages: 30 Total Word Count: ~8,500 words Report generated for the Virtual Chemistry Laboratory project.